অধ্যায় ৩

Operators এবং Decision Making

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Arithmetic Operator দিয়ে হিসাব করতে এবং Remainder-এর ব্যবহার বুঝতে।
- Assignment Operator দিয়ে Variable-এর মান সংক্ষেপে বদলাতে।
- Comparison ও Logical Operator দিয়ে শর্ত লিখতে।
- কোন মান Truthy এবং কোনটি Falsy, তা বলতে এবং এর ফাঁদ এড়াতে।
- `if`, `else if`, `else`, `switch` এবং Ternary Operator দিয়ে সিদ্ধান্ত নিতে।
- Operator Precedence বুঝে বন্ধনী দিয়ে হিসাবের ক্রম স্পষ্ট করতে।
- ব্যবহারকারীর দেওয়া মান ধাপে ধাপে যাচাই করতে।
- একটি সম্পূর্ণ **Shopping Discount Calculator** তৈরি ও পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ২ সম্পন্ন করা: `const` ও `let`, Data Type, `Number()` দিয়ে রূপান্তর, `===` এবং `null`।
- অধ্যায় ১-এর Function ও `if`-এর প্রাথমিক পরিচয়।
- `chapter-03` নামে একটি নতুন Folder।

এই অধ্যায় কেন গুরুত্বপূর্ণ

এ পর্যন্ত আপনার Program ওপর থেকে নিচে সোজা পথে চলেছে। বাস্তব Program-কে প্রতিনিয়ত সিদ্ধান্ত নিতে হয়: কেনাকাটা তিন হাজার টাকার বেশি হলে Delivery ফ্রি; Coupon সঠিক হলে ছাড়; পরিমাণ শূন্য হলে Order গ্রহণ করা হবে না। প্রতিটি সিদ্ধান্তের পেছনে থাকে একটি শর্ত, আর শর্ত লেখা হয় Operator দিয়ে।

এই অধ্যায়ের দুটি অংশ। প্রথম অংশে Operator: যেসব চিহ্ন দিয়ে হিসাব, তুলনা এবং একাধিক শর্ত জোড়া লাগানো হয়। দ্বিতীয় অংশে সিদ্ধান্ত গ্রহণ: শর্তের ফল অনুযায়ী Program-কে ভিন্ন পথে চালানো।

শেষে আপনি একটি দোকানের মূল্য-হিসাবের Program লিখবেন, যেখানে দাম, পরিমাণ, Coupon এবং সদস্যপদ মিলিয়ে চূড়ান্ত পরিশোধযোগ্য টাকা বের করা হয়।

৩.১ Expression এবং Operator

যে Code থেকে একটি মান পাওয়া যায়, তাকে **Expression** বলে। `5 + 3` একটি Expression, যার মান `8`। `price > 1000`-ও একটি Expression, যার মান `true` অথবা `false`।

Operator হলো সেই চিহ্ন, যা মান নিয়ে কাজ করে; আর যে মানগুলোর ওপর কাজ হয়, সেগুলো **Operand**। `5 + 3`-তে `+` Operator, `5` ও `3` Operand।

expression.js · JavaScript — চলবে: Browser Console ও Node.js

```
const unitPrice = 850;
const quantity = 3;
const subtotal = unitPrice * quantity;

console.log(subtotal);
console.log(subtotal >= 2000);
```

Output

```
2550
true
```

`unitPrice * quantity` একটি Expression; তার মান `subtotal`-এ রাখা হয়েছে। Expression যেকোনো জায়গায় বসানো যায়, যেখানে একটি মান দরকার: Variable-এ, `console.log()`-এর ভেতরে, অথবা শর্ত হিসেবে।

৩.২ Arithmetic Operator

Operator	কাজ	উদাহরণ	ফল
`+`	যোগ	`7 + 3`	`10`
`-`	বিয়োগ	`7 - 3`	`4`
`*`	গুণ	`7 * 3`	`21`
`/`	ভাগ	`7 / 2`	`3.5`
`%`	ভাগশেষ (Remainder)	`7 % 3`	`1`
`**`	ঘাত	`2 ** 3`	`8`

প্রথম চারটি পরিচিত। ভাগের ফল দশমিক হতে পারে; JavaScript-এ আলাদা "পূর্ণসংখ্যা ভাগ" নেই।

Remainder: ছোট Operator, অনেক কাজ

`%` ভাগ করার পর যা অবশিষ্ট থাকে, তা দেয়। এটি দিয়ে বেশ কিছু প্রশ্নের উত্তর সহজে পাওয়া যায়।

remainder.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log(10 % 2);
console.log(7 % 2);
console.log(125 % 60);
console.log(-7 % 3);
```

Output

```
0
1
5
-1
```

- কোনো সংখ্যাকে `2` দিয়ে ভাগ করে ভাগশেষ `0` হলে সংখ্যাটি জোড়; `1` হলে বিজোড়।
- ১২৫ মিনিটে পূর্ণ ঘণ্টা বাদ দিলে `125 % 60`, অর্থাৎ ৫ মিনিট থাকে।
- ঋণাত্মক সংখ্যার ভাগশেষ ঋণাত্মক হয়: `-7 % 3` এর ফল `-1`।

পূর্ণসংখ্যায় আনা

টাকার হিসাবে শতাংশ বের করলে দশমিক আসতে পারে। `Math.round()` সংখ্যাকে নিকটতম পূর্ণসংখ্যায় আনে; ঠিক মাঝামাঝি (`.5`) হলে ওপরের দিকে নেয়।

math-round.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log(Math.round(42.4));
console.log(Math.round(42.5));
console.log(Math.round(850 * 5 / 100));
```

Output

```
42
43
43
```

শেষ লাইনটি ৮৫০ টাকার ৫ শতাংশ। লক্ষ করুন, শতাংশকে `0.05` না লিখে `5 / 100` হিসেবে লেখা হয়েছে। অধ্যায় ২-এ দেখেছেন, দশমিক সংখ্যা হুবহু রাখা যায় না; আগে পূর্ণসংখ্যায় গুণ করে পরে ভাগ করলে সেই ঝুঁকি কমে।

৩.৩ Assignment Operator

`=` ডান পাশের মান বাঁ পাশের Variable-এ রাখে। প্রায়ই Variable-এর বর্তমান মানের সঙ্গে কিছু যোগ বা বিয়োগ করে আবার সেখানেই রাখতে হয়। এ কাজের সংক্ষিপ্ত রূপ আছে।

সংক্ষিপ্ত রূপ	পূর্ণ রূপ
`stock += 5`	`stock = stock + 5`
`stock -= 2`	`stock = stock - 2`
`stock *= 2`	`stock = stock * 2`
`stock /= 2`	`stock = stock / 2`
`stock++`	`stock = stock + 1`
`stock--`	`stock = stock - 1`

assignment.js · JavaScript — চলবে: Browser Console ও Node.js

```
let stock = 10;

stock += 5;
console.log(stock);

stock -= 2;
console.log(stock);

stock *= 2;
console.log(stock);

stock++;
console.log(stock);

stock--;
console.log(stock);
```

Output

```
15
13
26
27
26
```

মান বদলানো হচ্ছে, তাই `stock` অবশ্যই `let` দিয়ে তৈরি।

`++` এবং `--` Variable-এর আগে বা পরে বসানো যায়, এবং দুই ক্ষেত্রে আচরণ সামান্য আলাদা।

increment.js · JavaScript — চলবে: Browser Console ও Node.js

```
let a = 5;

console.log(a++);
console.log(a);
console.log(++a);
```

Output

```
5
6
7
```

`a++` আগে পুরোনো মান দেয়, তারপর বাড়ায়; `++a` আগে বাড়ায়, তারপর নতুন মান দেয়। এই সূক্ষ্ম পার্থক্য ভুলের উৎস। তাই `++` ও `--` সব সময় আলাদা লাইনে লিখুন, অন্য হিসাবের ভেতরে নয়।

৩.৪ Comparison Operator

Comparison Operator দুটি মান তুলনা করে এবং ফল হিসেবে একটি Boolean দেয়।

Operator	অর্থ	উদাহরণ	ফল
`===`	সমান (Type-সহ)	`5 === 5`	`true`
`!==`	সমান নয়	`5 !== 5`	`false`
`>`	বড়	`10 > 5`	`true`
`<`	ছোট	`10 < 5`	`false`
`>=`	বড় অথবা সমান	`10 >= 10`	`true`
`<=`	ছোট অথবা সমান	`10 <= 9`	`false`

সীমা নিয়ে কাজ করার সময় `>` এবং `>=`-এর পার্থক্যই সবচেয়ে বেশি ভুল ঘটায়। "৩০০০ টাকা বা তার বেশি হলে Delivery ফ্রি" মানে `>= 3000`; `> 3000` লিখলে ঠিক ৩০০০ টাকার ক্রেতা সুবিধাটি পাবেন না।

String-ও তুলনা করা যায়, তবে নিয়মটি আলাদা।

compare-strings.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log('apple' < 'banana');
console.log(10 < 9);
console.log('10' < '9');
```

Output

```
true
false
true
```

String তুলনা হয় অভিধানের মতো, বাঁ দিক থেকে অক্ষর ধরে ধরে। `'10'` এবং `'9'`-এর প্রথম অক্ষর `'1'` ও `'9'`; `'1'` আগে আসে, তাই `'10' < '9'` সত্য। সংখ্যা হিসেবে এটি ভুল। এ কারণেই অধ্যায় ২-এর নিয়মটি গুরুত্বপূর্ণ: তুলনা বা হিসাবের আগে String-কে `Number()` দিয়ে সংখ্যায় রূপান্তর করুন।

৩.৫ Logical Operator

একাধিক শর্ত জোড়া লাগাতে বা একটি শর্ত উল্টে দিতে Logical Operator ব্যবহার করা হয়।

Operator	নাম	কখন সত্য
`&&`	AND (এবং)	দুই পাশই সত্য হলে
`||`	OR (অথবা)	অন্তত এক পাশ সত্য হলে
`!`	NOT (নয়)	মানটি মিথ্যা হলে

logical.js · JavaScript — চলবে: Browser Console ও Node.js

```
const age = 20;
const hasId = true;
const isWeekend = false;
const isHoliday = true;

console.log(age >= 18 && hasId);
console.log(isWeekend || isHoliday);
console.log(!hasId);
console.log(age >= 18 && !isWeekend);
```

Output

```
true
true
false
true
```

- প্রথম লাইন: বয়স ১৮ বা বেশি **এবং** পরিচয়পত্র আছে—দুটোই সত্য, তাই ফল সত্য।
- দ্বিতীয় লাইন: সপ্তাহান্ত **অথবা** ছুটির দিন—একটি সত্য হলেই যথেষ্ট।
- তৃতীয় লাইন: `!` সত্যকে মিথ্যা এবং মিথ্যাকে সত্য করে।

অধ্যায় ২-এর Project-এ একই দুই লাইন তিনটি আলাদা `if`-এ লিখতে হয়েছিল। এখন সেটি একটি শর্তে লেখা যায়: `bangla < 33 || english < 33 || math < 33`।

Short-Circuit: প্রয়োজন না হলে থেমে যাওয়া

`&&`-এর বাঁ পাশ মিথ্যা হলে পুরো ফল মিথ্যা হবেই, তাই JavaScript ডান পাশ আর দেখে না। একইভাবে `||`-এর বাঁ পাশ সত্য হলে ডান পাশ দেখা হয় না। এই আচরণকে **Short-Circuit** বলে।

আরেকটি বিষয়: `&&` এবং `||` সব সময় `true` বা `false` ফেরত দেয় না; যে Operand-এ গিয়ে থামে, সেটির মানই ফেরত দেয়।

short-circuit.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log('' || 'Guest');
console.log('Sara' || 'Guest');
console.log(0 || 10);
console.log(0 ?? 10);
console.log(null ?? 10);
```

Output

```
Guest
Sara
10
0
10
```

`||` দিয়ে একটি বিকল্প মান দেওয়া যায়: নাম ফাঁকা হলে `'Guest'`। কিন্তু তৃতীয় লাইনে সমস্যা: `0` একটি বৈধ মান হতে পারে, অথচ `||` সেটিকে বাদ দিয়ে `10` দিয়েছে। এ জন্য `??` আছে, যা শুধু `null` বা `undefined` হলেই বিকল্পটি নেয়। শূন্য বা ফাঁকা String যেখানে বৈধ মান, সেখানে `??` ব্যবহার করুন।

৩.৬ Truthy এবং Falsy

শর্তের জায়গায় Boolean ছাড়া অন্য মান বসালে JavaScript সেটিকে নিজে থেকে সত্য বা মিথ্যা হিসেবে ধরে নেয়। যে মানগুলো মিথ্যা হিসেবে গণ্য হয়, সেগুলো **Falsy**; বাকি সব **Truthy**।

Falsy মান মাত্র কয়েকটি, তাই এগুলো মনে রাখাই সহজ:

Falsy মান	অর্থ
`false`	মিথ্যা
`0`	শূন্য
`''`	ফাঁকা String

Falsy মান	অর্থ
`null`	ইচ্ছাকৃতভাবে কিছু নেই
`undefined`	মান দেওয়া হয়নি
`NaN`	অর্থহীন সংখ্যা

truthy-falsy.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log(Boolean(0));
console.log(Boolean(''));
console.log(Boolean(null));
console.log(Boolean('0'));
console.log(Boolean('false'));
console.log(Boolean(' '));
console.log(Boolean({}));
```

Output

```
false
false
false
true
true
true
true
```

শেষ চারটি সত্য। `'0'` ও `'false'` ফাঁকা নয় এমন String; একটি ফাঁকা জায়গাও একটি অক্ষর; আর যেকোনো Object, এমনকি ফাঁকা Object-ও, Truthy।

Truthy-র ফাঁদ

zero-trap.js · JavaScript — চলবে: Browser Console ও Node.js

```
function describeMark(mark) {
  if (mark) {
    return `নম্বর: ${mark}`;
  }

  return 'নম্বর দেওয়া হয়নি';
}

console.log(describeMark(75));
console.log(describeMark(0));
```

Output

```
নম্বর: 75
নম্বর দেওয়া হয়নি
```

শূন্য একটি বৈধ নম্বর, কিন্তু `if (mark)` সেটিকে "নেই" হিসেবে ধরেছে, কারণ `0` Falsy। আপনি যা জানতে চান, ঠিক সেটিই লিখুন: নম্বর দেওয়া হয়েছে কি না জানতে চাইলে `mark !== null`।

> **মনে রাখুন:** সংখ্যা বা String-এর ক্ষেত্রে `if (value)` লেখার আগে ভাবুন: শূন্য বা ফাঁকা String কি এখানে বৈধ মান হতে পারে? হলে স্পষ্ট তুলনা লিখুন।

৩.৭ if, else if এবং else

`if` শর্ত সত্য হলে একটি Block চালায়। শর্ত মিথ্যা হলে অন্য কিছু করতে চাইলে `else` যোগ করুন; মাঝখানে আরও শর্ত থাকলে `else if`।

shipping.js · JavaScript — চলবে: Browser Console ও Node.js

```javascript
function shippingMessage(amount) {
  if (amount >= 5000) {
    return 'Delivery ফ্রি';
  } else if (amount >= 2000) {
    return 'Delivery-তে অর্ধেক ছাড়';
  } else {
    return 'সাধারণ Delivery';
  }
}

console.log(shippingMessage(6000));
console.log(shippingMessage(2500));
console.log(shippingMessage(500));
```

Output

```
Delivery ফ্রি
Delivery-তে অর্ধেক ছাড়
সাধারণ Delivery
```

JavaScript শর্তগুলো ওপর থেকে নিচে যাচাই করে এবং **প্রথম যেটি সত্য হয়, শুধু সেই Block-টি চালায়**। বাকিগুলো আর দেখা হয় না। কোনোটিই সত্য না হলে `else`-এর Block চলে।

শর্তের ক্রম গুরুত্বপূর্ণ

wrong-order.js · JavaScript — চলবে: Browser Console ও Node.js

```
function wrongMessage(amount) {
  if (amount >= 2000) {
    return 'Delivery-তে অর্ধেক ছাড়';
  } else if (amount >= 5000) {
    return 'Delivery ফ্রি';
  } else {
    return 'সাধারণ Delivery';
  }
}

console.log(wrongMessage(6000));
```

Output

```
Delivery-তে অর্ধেক ছাড়
```

৬০০০ টাকার ক্রেতা ফ্রি Delivery পাওয়ার কথা, কিন্তু `6000 >= 2000` সত্য হওয়ায় প্রথম Block-ই চলেছে, আর দ্বিতীয় শর্তে কখনো পৌঁছানো হয়নি। সীমা নিয়ে কাজ করার সময় সবচেয়ে কঠোর শর্তটি সবার আগে লিখুন।

৩.৮ switch

একটি মানকে অনেকগুলো নির্দিষ্ট মানের সঙ্গে মেলাতে হলে লম্বা `else if`-এর চেয়ে `switch` পড়তে সহজ।

payment.js · JavaScript — চলবে: Browser Console ও Node.js

```
function paymentLabel(method) {
  switch (method) {
    case 'card':
      return 'Card-এ পরিশোধ';
    case 'cash':
      return 'পণ্য হাতে পেয়ে পরিশোধ';
    case 'mobile':
      return 'Mobile Banking';
    default:
      return 'অজানা পদ্ধতি';
  }
}

console.log(paymentLabel('cash'));
console.log(paymentLabel('cheque'));
```

Output

```
পণ্য হাতে পেয়ে পরিশোধ
অজানা পদ্ধতি
```

`switch` বন্ধনীর মানটিকে প্রতিটি `case`-এর সঙ্গে `===` দিয়ে মেলায়। যেটি মেলে, সেখান থেকে Code চলা শুরু হয়। কোনোটি না মিললে `default` চলে। `default` সব সময় লিখুন, যাতে অপ্রত্যাশিত মান নীরবে পার না হয়ে যায়।

break এবং Fallthrough

ওপরের উদাহরণে প্রতিটি `case` `return` দিয়ে শেষ হয়েছে। `return` না থাকলে প্রতিটি `case`-এর শেষে `break` লিখতে হয়; না লিখলে JavaScript থামে না, পরের `case`-এর Code-ও চালিয়ে যায়। একে **Fallthrough** বলে।

fallthrough.js · JavaScript — চলবে: Browser Console ও Node.js

```
function dayType(day) {
  let type = 'কর্মদিবস';

  switch (day) {
    case 'fri':
    case 'sat':
      type = 'সাপ্তাহিক ছুটি';
      break;
    case 'sun':
      type = 'সপ্তাহের প্রথম কর্মদিবস';
      break;
    default:
      break;
  }

  return type;
}

console.log(dayType('fri'));
console.log(dayType('sat'));
console.log(dayType('mon'));
```

Output

```
সাপ্তাহিক ছুটি
সাপ্তাহিক ছুটি
কর্মদিবস
```

এখানে Fallthrough ইচ্ছা করে ব্যবহার করা হয়েছে: `'fri'`-এর নিজস্ব কোনো Code নেই, তাই সেটি `'sat'`-এর Code-এ গড়িয়ে পড়ে। একাধিক মানের জন্য একই কাজ করতে এটি একটি পরিচ্ছন্ন উপায়। কিন্তু ভুল করে `break` বাদ পড়লে একই আচরণ Bug হয়ে দাঁড়ায়।

‘‘

	if / else if	switch
উপযুক্ত	সীমা, পরিসর, জটিল শর্ত (`>=`, `&&`)	একটি মানকে নির্দিষ্ট কিছু মানের সঙ্গে মেলানো
তুলনা	যেকোনো Expression	শুধু `===`

৩.৯ Ternary Operator

দুটি মানের মধ্যে একটি বেছে নেওয়ার সংক্ষিপ্ত উপায় **Ternary Operator**। এর তিনটি অংশ: শর্ত, সত্য হলে মান, মিথ্যা হলে মান।

ternary.js · JavaScript — চলবে: Browser Console ও Node.js

```
const stock = 0;
const badge = stock > 0 ? 'মজুত আছে' : 'শেষ হয়ে গেছে';

console.log(badge);

const isMember = true;
const discountPercent = isMember ? 5 : 0;

console.log(discountPercent);
```

Output

```
শেষ হয়ে গেছে
5
```

`শর্ত ? মান১ : মান২` —শর্ত সত্য হলে পুরো Expression-এর মান `মান১`, নইলে `মান২`। এটি একটি Expression, তাই সরাসরি `const`-এ রাখা যায়; `if` / `else` দিয়ে একই কাজ করতে `let` এবং চার-পাঁচ লাইন লাগত।

Ternary ব্যবহার করুন শুধু সরল, দুই-পথের সিদ্ধান্তে। একটি Ternary-র ভেতরে আরেকটি বসালে Code পড়া কঠিন হয়ে যায়; সে ক্ষেত্রে `if` / `else` লিখুন।

৩.১০ Operator Precedence

একটি Expression-এ একাধিক Operator থাকলে কোনটি আগে হিসাব হবে, তা ঠিক করে **Operator Precedence**। গণিতের মতোই গুণ ও ভাগ যোগ-বিয়োগের আগে হয়।

precedence.js · JavaScript — চলবে: Browser Console ও Node.js

```
console.log(2 + 3 * 4);
console.log((2 + 3) * 4);
console.log(10 - 4 - 3);
console.log(2 ** 3 ** 2);
console.log(true || false && false);
console.log((true || false) && false);
```

Output

```
14
20
3
512
true
false
```

এই অধ্যায়ের Operator-গুলোর ক্রম, আগে থেকে পরে:

ক্রম	Operator
১	`( )` বন্ধনী
২	`!`, `++`, `--`
৩	`**`
৪	`*`, `/`, `%`
৫	`+`, `-`
৬	`<`, `<=`, `>`, `>=`
৭	`===`, `!==`
৮	`&&`
৯	`||`, `??`
১০	`? :` (Ternary)
১১	`=`, `+=`, `-=` ইত্যাদি

একই স্তরের Operator সাধারণত বাঁ থেকে ডানে হিসাব হয়: `10 - 4 - 3` মানে `(10 - 4) - 3`। ব্যতিক্রম `**`, যা ডান থেকে বাঁয়ে: `2 ** 3 ** 2` মানে `2 ** 9`।

সবচেয়ে গুরুত্বপূর্ণ সারি দুটি হলো ৮ ও ৯: `&&` সব সময় `||`-এর আগে হিসাব হয়। তাই `true || false && false`-এ আগে `false && false` হয়, তারপর `true || false`।

> **পরামর্শ:** এই তালিকা মুখস্থ করার প্রয়োজন নেই। যেখানে `&&` এবং `||` একসঙ্গে আছে, অথবা ক্রম নিয়ে সামান্য সন্দেহও আছে, সেখানে বন্ধনী দিন। বন্ধনী Code-কে ধীর করে না, কিন্তু পাঠকের ভুল বোঝা আটকায়।

৩.১১ Input Validation

ব্যবহারকারীর দেওয়া মান ব্যবহারের আগে যাচাই করাকে **Input Validation** বলে। অধ্যায় ২-এ এটি একাধিক আলাদা `if` দিয়ে করা হয়েছিল। Logical Operator দিয়ে একই কাজ আরও সংক্ষেপে হয়।

validate-quantity.js · JavaScript — চলবে: Browser Console ও Node.js

```
function isValidQuantity(text) {
  const clean = text.trim();

  if (clean === '') {
    return false;
  }

  const quantity = Number(clean);

  return Number.isInteger(quantity) && quantity >= 1 && quantity <= 100;
}

console.log(isValidQuantity('5'));
console.log(isValidQuantity(''));
console.log(isValidQuantity('0'));
console.log(isValidQuantity('2.5'));
console.log(isValidQuantity('abc'));
console.log(isValidQuantity('101'));
```

Output

```
true
false
false
false
false
false
```

শেষ `return`-এ তিনটি শর্ত `&&` দিয়ে জোড়া: পূর্ণসংখ্যা হতে হবে, **এবং** ১ বা বেশি, **এবং** ১০০ বা কম। Comparison-এর ফল নিজেই একটি Boolean, তাই সেটি সরাসরি ফেরত দেওয়া যায়; `if (...) { return true; } else { return false; }` লেখার দরকার নেই।

Guard Clause: আগে সমস্যাগুলো সরিয়ে ফেলা

Function-এর শুরুতেই অগ্রহণযোগ্য অবস্থাগুলো যাচাই করে `return` করে দেওয়ার ধরনটিকে **Guard Clause** বলে। ওপরের `if (clean === '')` একটি Guard Clause। এতে মূল হিসাব `if`-এর ভেতরে ঢুকে যায় না, আর Function-এর নিচের অংশে নিশ্চিত থাকা যায় যে মানগুলো ঠিক আছে।

যাচাইয়ের একটি নির্ভরযোগ্য ক্রম:

১. মানটি আছে কি না (ফাঁকা নয়)।
২. সঠিক Type-এ রূপান্তর করা যায় কি না।
৩. মানটি অনুমোদিত সীমার মধ্যে কি না।
৪. অন্য মানের সঙ্গে মিলিয়ে নিয়মটি মানছে কি না (যেমন: এই Coupon-এর জন্য ন্যূনতম কেনাকাটা)।

> **সতর্কতা:** Browser বা ব্যবহারকারীর Computer-এ করা যাচাই শুধু সুবিধার জন্য। দাম, ছাড় বা টাকার হিসাব সব সময় Server-এ আবার করতে হয়, কারণ ব্যবহারকারীর দিকের Code বদলে ফেলা সম্ভব। বিষয়টি অধ্যায় ২০-এ বিস্তারিত আছে।

৩.১২ সাধারণ ভুল

ভুল	কী দেখা যায়	সমাধান
শর্তে `===`-এর বদলে `=` লেখা	শর্ত সব সময় সত্য, আর Variable-এর মান বদলে যায়	তুলনার জন্য `===`
`>` এবং `>=` গুলিয়ে ফেলা	ঠিক সীমার মানে ভুল ফল	নিয়মটি "বা তার বেশি" কি না পড়ে দেখুন; সীমার মান দিয়ে পরীক্ষা করুন
`else if`-এর ভুল ক্রম	পরের শর্তে কখনো পৌঁছানো যায় না	কঠোরতম শর্ত আগে
`switch`-এ `break` বাদ পড়া	পরের `case`-এর Code-ও চলে	প্রতিটি `case` `break` বা `return` দিয়ে শেষ করুন
`if (value)` দিয়ে শূন্য যাচাই	বৈধ `0` বাদ পড়ে	`value !== null` বা স্পষ্ট তুলনা
`||` দিয়ে বিকল্প মান, যেখানে `0` বৈধ	`0`-এর জায়গায় বিকল্প বসে	`??` ব্যবহার করুন
String হিসেবে সংখ্যা তুলনা	`'10' < '9'` সত্য	আগে `Number()`
`&&` ও `||` বন্ধনী ছাড়া মেশানো	হিসাবের ক্রম প্রত্যাশামতো নয়	বন্ধনী দিন
`1 < x < 10` লেখা	সব সময় সত্য	`x > 1 && x < 10`

প্রথম ভুলটি চালিয়ে দেখা শিক্ষণীয়।

assign-in-condition.js · JavaScript চলবে: Browser Console ও Node.js

```
let coupon = 'NONE';

if (coupon = 'SAVE10') {
  console.log('Coupon প্রয়োগ হয়েছে');
}

console.log(coupon);
```

Output

```
Coupon প্রয়োগ হয়েছে
SAVE10
```

`coupon = 'SAVE10'` তুলনা নয়, Assignment। এটি `coupon`-এ `'SAVE10'` রাখে, আর Assignment-এর মান হয় সেই String, যা Truthy। ফলে শর্ত সব সময় সত্য, এবং Variable-টিও নীরবে বদলে গেছে। Coupon যাচাইয়ের মতো জায়গায় এই একটি অক্ষরের ভুল মানে সবাইকে ছাড় দেওয়া।

৩.১৩ Debugging নির্দেশিকা

শর্ত-সংক্রান্ত Bug-এ Program কোনো Error দেখায় না; শুধু ভুল পথে যায়। তাই প্রশ্ন হলো: শর্তটির মান আসলে কী হচ্ছে?

একটি জটিল শর্তকে ভেঙে প্রতিটি অংশ আলাদাভাবে দেখুন।

debug-condition.js · JavaScript চলবে: Browser Console ও Node.js

```
const subtotal = 4800;
const code = 'SAVE20';

console.log('code মিলেছে:', code === 'SAVE20');
console.log('ন্যূনতম পূরণ:', subtotal >= 5000);
console.log('পুরো শর্ত:', code === 'SAVE20' && subtotal >= 5000);
```

Output

```
code মিলেছে: true
ন্যূনতম পূরণ: false
পুরো শর্ত: false
```

এক নজরে বোঝা যাচ্ছে কোন অংশটি শর্তকে মিথ্যা করছে।

Program ভুল পথে গেলে এই ক্রমে যাচাই করুন:

১. শর্তের প্রতিটি অংশের মান আলাদাভাবে `console.log()` করুন।

২. তুলনার দুই পাশের `typeof` দেখুন; একটি String আর একটি Number নয় তো?
৩. ঠিক সীমার মান দিয়ে পরীক্ষা করুন: সীমাটি, তার এক কম এবং এক বেশি।
৪. `else if`-এর ক্রম দেখুন: ওপরের কোনো শর্ত কি নিচেরটিকে ঢেকে দিচ্ছে?
৫. `switch`-এ প্রতিটি `case`-এর শেষে `break` বা `return` আছে কি না দেখুন।
৬. শর্তে `=` আর `===` মিলিয়ে দেখুন।
৭. প্রতিটি Block-এর শুরুতে একটি `console.log()` বসিয়ে দেখুন Program আসলে কোন পথে ঢুকছে।

৩.১৪ পেশাদার অভ্যাস

- **আপনি যা বোঝাতে চান, ঠিক তা-ই লিখুন।** `if (count)` নয়, `if (count > 0)`।
- **Guard Clause দিয়ে শুরু করুন।** অগ্রহণযোগ্য অবস্থাগুলো আগে সরিয়ে দিলে মূল Logic সমতল ও পাঠযোগ্য থাকে।
- **গভীর Nesting এড়িয়ে চলুন।** একটি `if`-এর ভেতরে আরেকটি, তার ভেতরে আরেকটি—এমন হলে শর্তগুলো `&&` দিয়ে জোড়া লাগান অথবা আলাদা Function-এ নিন।
- **জটিল শর্তকে নাম দিন।** `const qualifiesForFreeDelivery = payable >= 3000;` লিখে তারপর `if (qualifiesForFreeDelivery)`।
- **সন্দেহ হলে বন্ধনী।**
- **`switch`-এ সব সময় `default`।**
- **সীমার সংখ্যাগুলোকে নাম দিন।** `3000` বা `120` Code-এর মাঝখানে না লিখে `const` করুন।
- **`++` ও `--` আলাদা লাইনে।**
- **ব্যবসার নিয়ম আগে ভাষায় লিখে নিন।** "Coupon-এর ছাড় আগে, সদস্য-ছাড় পরে"—ক্রম বদলালে টাকার অঙ্ক বদলায়।

৩.১৫ Project: Shopping Discount Calculator

একটি Online দোকানের মূল্য-হিসাবের অংশটি লিখবেন। দাম ও পরিমাণ Text হিসেবে আসবে; সঙ্গে থাকবে একটি Coupon Code এবং ক্রেতা সদস্য কি না।

Project-এর চাহিদা

এগুলো এই শিক্ষামূলক Project-এর জন্য তৈরি নিয়ম।

- দাম শূন্য বা তার বেশি পূর্ণসংখ্যা (টাকায়)। পরিমাণ ১ থেকে ১০০-এর মধ্যে পূর্ণসংখ্যা।
- Coupon `SAVE10` দিলে ১০ শতাংশ ছাড়। `SAVE20` দিলে ২০ শতাংশ ছাড়, তবে মোট কেনাকাটা কমপক্ষে ৫০০০ টাকা হতে হবে।

- Coupon Code ছোট বা বড় হাতের অক্ষরে এবং আগে-পরে ফাঁকা জায়গাসহ লেখা হলেও গৃহীত হবে। Coupon না দিলে কোনো ছাড় নেই। অচেনা Code দিলে তা জানাতে হবে; নীরবে উপেক্ষা করা যাবে না।
- সদস্য হলে Coupon-এর ছাড়ের পর অবশিষ্ট টাকার ওপর আরও ৫ শতাংশ ছাড়।
- প্রতিটি ছাড় নিকটতম পূর্ণ টাকায় আনা হবে।
- সব ছাড়ের পর টাকার অঙ্ক ৩০০০ বা তার বেশি হলে Delivery ফ্রি; নইলে ১২০ টাকা।

File-এর কাঠামো

Folder-এর কাঠামো

```
shopping-discount-calculator/
├── discount.js
├── README.md
└── tests/
    └── discount.test.js
```

discount.js

File-টি তিন ধাপে দেখানো হলো। তিনটি অংশ পরপর একই File-এ লিখুন; `// ...` লাইনগুলো লিখতে হবে না।

ধাপ ১: স্থির মান এবং Text থেকে পূর্ণসংখ্যা

discount.js · JavaScript নির্বাচিত অংশ (Selected Excerpt) চলবে: Node.js

```javascript
'use strict';

const DELIVERY_FEE = 120;
const FREE_DELIVERY_FROM = 3000;
const MEMBER_PERCENT = 5;
const SAVE20_MINIMUM = 5000;

// Text থেকে শূন্য বা তার বেশি পূর্ণসংখ্যা তৈরি করে; না পারলে null।
function toWholeNumber(text) {
  const clean = text.trim();

  if (clean === '') {
    return null;
  }

  const value = Number(clean);

  if (!Number.isInteger(value) || value < 0) {
    return null;
  }

  return value;
}
```

```
// ...
```

চারটি সংখ্যাকে শুরুতেই নাম দেওয়া হয়েছে। দোকান Delivery-র খরচ বদলালে একটি লাইন বদলালেই চলবে। `toWholeNumber` অধ্যায় ২-এর `toMark`-এর মতোই, তবে শেষ দুটি যাচাই এখন `||` দিয়ে এক শর্তে: পূর্ণসংখ্যা **নয়, অথবা** ঋণাত্মক—যেকোনো একটি হলেই বাতিল।

ধাপ ২: Coupon থেকে ছাড়ের শতাংশ

discount.js · JavaScript নির্বাচিত অংশ (Selected Excerpt) চলবে: Node.js

```
// ...

// Coupon Code থেকে ছাড়ের শতাংশ দেয়; অচেনা Code হলে null।
function getCouponPercent(code) {
  switch (code) {
    case '':
      return 0;
    case 'SAVE10':
      return 10;
    case 'SAVE20':
      return 20;
    default:
      return null;
  }
}

// ...
```

একটি মানকে কয়েকটি নির্দিষ্ট মানের সঙ্গে মেলানো— `switch`-এর আদর্শ ব্যবহার। ফাঁকা Code বৈধ এবং তার ছাড় শূন্য; অচেনা Code-এর জন্য `default` থেকে `null`। শূন্য শতাংশ একটি বৈধ উত্তর, তাই "অচেনা" বোঝাতে `0` ব্যবহার করা যেত না।

ধাপ ৩: পুরো হিসাব

discount.js · JavaScript নির্বাচিত অংশ (Selected Excerpt) চলবে: Node.js

```
// ...

// দাম, পরিমাণ, Coupon ও সদস্যপদ থেকে হিসাবের একটি বাক্য তৈরি করে।
function calculateOrder(priceText, quantityText, couponText, isMember) {
  const price = toWholeNumber(priceText);
  const quantity = toWholeNumber(quantityText);

  if (price === null) {
    return 'দাম সঠিক নয়।';
  }
```

```
  if (quantity === null || quantity < 1 || quantity > 100) {
    return 'পরিমাণ ১ থেকে ১০০-এর মধ্যে হতে হবে।';
  }

  const subtotal = price * quantity;
  const code = couponText.trim().toUpperCase();
  const couponPercent = getCouponPercent(code);

  if (couponPercent === null) {
    return 'Coupon Code সঠিক নয়।';
  }

  if (code === 'SAVE20' && subtotal < SAVE20_MINIMUM) {
    return 'SAVE20-এর জন্য কমপক্ষে 5000 টাকার কেনাকাটা প্রয়োজন।';
  }

  const couponDiscount = Math.round(subtotal * couponPercent / 100);
  const afterCoupon = subtotal - couponDiscount;
  const memberPercent = isMember ? MEMBER_PERCENT : 0;
  const memberDiscount = Math.round(afterCoupon * memberPercent / 100);
  const payable = afterCoupon - memberDiscount;
  const delivery = payable >= FREE_DELIVERY_FROM ? 0 : DELIVERY_FEE;
  const discount = couponDiscount + memberDiscount;
  const total = payable + delivery;

  return `মোট ${subtotal}, ছাড় ${discount}, Delivery ${delivery}, দিতে হবে ${total}`;
}

console.log(calculateOrder('1500', '2', 'SAVE10', false));
console.log(calculateOrder('2500', '3', ' save20 ', true));
console.log(calculateOrder('850', '1', '', true));
console.log(calculateOrder('1200', '2', 'SAVE20', false));
console.log(calculateOrder('1200', '0', '', false));
console.log(calculateOrder('1200', '2', 'FREE50', false));
```

Output

```
মোট 3000, ছাড় 300, Delivery 120, দিতে হবে 2820
মোট 7500, ছাড় 1800, Delivery 0, দিতে হবে 5700
মোট 850, ছাড় 43, Delivery 120, দিতে হবে 927
SAVE20-এর জন্য কমপক্ষে 5000 টাকার কেনাকাটা প্রয়োজন।
পরিমাণ ১ থেকে ১০০-এর মধ্যে হতে হবে।
Coupon Code সঠিক নয়।
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **চারটি Guard Clause।** দাম, পরিমাণ, Coupon-এর অস্তিত্ব এবং Coupon-এর শর্ত—অনুচ্ছেদ ৩.১১-এর ক্রমেই যাচাই করা হয়েছে। এরপরের অংশে সব মান নিশ্চিতভাবে সঠিক।

- **`quantity === null || quantity < 1 || quantity > 100`** — `||`-এর Short-Circuit এখানে কাজে লাগছে: `quantity` `null` হলে বাকি দুটি তুলনা আর করা হয় না।
- **`couponText.trim().toUpperCase()`** — `toUpperCase()` String-এর সব অক্ষর বড় হাতের করে দেয়। ফলে `' save20 '` এবং `'SAVE20'` একই Code হিসেবে গণ্য হয়।
- **`code === 'SAVE20' && subtotal < SAVE20_MINIMUM`** — নিয়মটি দুটি মান মিলিয়ে: Code-টি `SAVE20` **এবং** কেনাকাটা সীমার নিচে।
- **দুটি Ternary** — সদস্য হলে ৫ শতাংশ, নইলে শূন্য; পরিশোধযোগ্য অঙ্ক সীমা ছুঁলে Delivery শূন্য, নইলে ১২০। দুটিই সরল দুই-পথের সিদ্ধান্ত।
- **হিসাবের ক্রম।** Coupon-এর ছাড় পুরো অঙ্কের ওপর; সদস্য-ছাড় তার পরের অঙ্কের ওপর। দ্বিতীয় উদাহরণে ৭৫০০ থেকে ২০ শতাংশ (১৫০০) বাদ গিয়ে ৬০০০, তার ৫ শতাংশ (৩০০) বাদ গিয়ে ৫৭০০।
- **`Math.round()`** — তৃতীয় উদাহরণে ৮৫০-এর ৫ শতাংশ ৪২.৫, যা ৪৩ টাকায় আনা হয়েছে।

প্রথম উদাহরণটি লক্ষ করার মতো: ৩০০০ টাকার কেনাকাটায় ছাড়ের পর থাকে ২৭০০, যা ফ্রি Delivery-র সীমার নিচে। নিয়মে বলা আছে "সব ছাড়ের পর", তাই ১২০ টাকা যোগ হয়েছে।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
node discount.js
```

দাম, পরিমাণ, Coupon, সদস্য	যা দেখা যাওয়ার কথা
`'1500', '2', 'SAVE10', false`	মোট 3000, ছাড় 300, Delivery 120, দিতে হবে 2820
`'2500', '3', ' save20 ', true`	মোট 7500, ছাড় 1800, Delivery 0, দিতে হবে 5700
`'850', '1', '', true`	মোট 850, ছাড় 43, Delivery 120, দিতে হবে 927
`'1200', '2', 'SAVE20', false`	SAVE20-এর জন্য কমপক্ষে 5000 টাকার কেনাকাটা প্রয়োজন।
পরিমাণ `'0'`, `'101'`, `'2.5'` বা ফাঁকা	পরিমাণ ১ থেকে ১০০-এর মধ্যে হতে হবে।
অচেনা Coupon, যেমন `'FREE50'`	Coupon Code সঠিক নয়।

Companion Source Code-এ এই Project-এর ১৬টি Automated Test আছে; প্রতিটি সীমার মান (যেমন ঠিক ৩০০০ এবং ২৯৯৯) আলাদাভাবে পরীক্ষা করা হয়েছে। চালানোর Command `README.md`-তে আছে।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
ঠিক ৩০০০ টাকায় Delivery ফ্রি হয় না	`>=`-এর জায়গায় `>`	`payable >= FREE_DELIVERY_FROM`

লক্ষণ	সম্ভাব্য কারণ	করণীয়
ছোট হাতের `save10` বাতিল হয়	`toUpperCase()` নেই	`couponText.trim().toUpperCase()`
সব Coupon-এ একই ছাড়	`switch`-এ `return` বা `break` বাদ পড়েছে	প্রতিটি `case` `return` দিয়ে শেষ করুন
সদস্য-ছাড় বেশি আসছে	ছাড়টি `subtotal`-এর ওপর হিসাব হচ্ছে	`afterCoupon`-এর ওপর হিসাব করুন
ছাড়ে দশমিক সংখ্যা	`Math.round()` নেই	প্রতিটি ছাড় `Math.round()` করুন
ফাঁকা পরিমাণে `TypeError`	`null`-এর যাচাই পরে লেখা	`quantity === null` শর্তটি সবার আগে

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project এবং নিয়মগুলো উদাহরণ মাত্র। এখানে একটিই পণ্য হিসাব করা হয়; একাধিক পণ্যের Cart অধ্যায় ৪-এ আসবে। Coupon-এর তালিকা Code-এর ভেতরে লেখা। সবচেয়ে গুরুত্বপূর্ণ: বাস্তব দোকানে মূল্য ও ছাড়ের চূড়ান্ত হিসাব Server-এ হতে হয়; ক্রেতার Computer-এ করা হিসাব শুধু আগাম ধারণা দেওয়ার জন্য।

৩.১৬ অনুশীলনী

প্রাথমিক স্তর

১. না চালিয়ে বলুন: `17 % 5`, `2 + 3 * 2` এবং `(2 + 3) * 2`-এর ফল কত। তারপর চালিয়ে মিলিয়ে নিন।

২. `isEven` নামে একটি Function লিখুন, যা সংখ্যা জোড় হলে `true` এবং বিজোড় হলে `false` ফেরত দেয়।

৩. `let points = 50;` দিয়ে শুরু করে সংক্ষিপ্ত Assignment Operator ব্যবহার করে ১০ যোগ করুন, তারপর দ্বিগুণ করুন, তারপর ১ কমান। প্রতিটি ধাপের পর মান দেখান।

৪. না চালিয়ে বলুন কোনগুলো Truthy: `'0'`, `0`, `' '`, `''`, `null`, `'null'`।

৫. নিচের Function-এ একটি ভুল আছে, যার কারণে `canVote(15)`-ও `true` দেয়। ভুলটি খুঁজে ঠিক করুন।

can-vote.js · JavaScript চলবে: Browser Console ও Node.js

```
function canVote(age) {
  if (age >= 18 || age < 18) {
    return true;
  }

  return false;
}

console.log(canVote(15));
```

মধ্যম স্তর

৬. `getGrade` নামে একটি Function লিখুন, যা `else if` ব্যবহার করে নম্বর থেকে Grade দেয়: ৮০ বা বেশি `A+`, ৬০ বা বেশি `A`, ৩৩ বা বেশি `B`, নইলে `F`।

৭. `seasonOf` নামে একটি Function লিখুন, যা `switch` ব্যবহার করে মাসের নম্বর (১–১২) থেকে ঋতু দেয়: ১২, ১, ২ হলে `শীত`; ৩, ৪, ৫ হলে `গ্রীষ্ম`; ৬, ৭, ৮, ৯ হলে `বর্ষা`; ১০, ১১ হলে `হেমন্ত`; অন্য কিছু হলে `অজানা`।

৮. `ticketPrice` নামে একটি Function লিখুন: বয়স ৫-এর কম হলে `0`, ৫ থেকে ১২ হলে `50`, বাকিদের `100`। তারপর সদস্য হলে দাম অর্ধেক করুন (Ternary ব্যবহার করুন)।

৯. `isValidAge` নামে একটি Function লিখুন, যা Text নিয়ে `true` দেয় শুধু তখনই, যখন সেটি ০ থেকে ১২০-এর মধ্যে পূর্ণসংখ্যা। একটিমাত্র `return`-এ `&&` ব্যবহার করুন (ফাঁকা যাচাইয়ের Guard Clause বাদে)।

চ্যালেঞ্জ স্তর

১০. **নতুন Coupon:** `FLAT200` নামে একটি Coupon-এর ছাড় হিসাব করার Function লিখুন: কেনাকাটা ১০০০ টাকা বা বেশি হলে সরাসরি ২০০ টাকা ছাড়, নইলে শূন্য।

১১. **Delivery-র ধাপ:** `deliveryFee` Function লিখুন: অঙ্ক ৩০০০ বা বেশি হলে `0`; ১৫০০ বা বেশি হলে `60`; নইলে `120`। ঠিক সীমার মানগুলো দিয়ে পরীক্ষা করুন।

১২. **সর্বোচ্চ ছাড়:** `cappedDiscount` Function লিখুন, যা শতাংশ-ছাড় হিসাব করে, কিন্তু ছাড় কখনো ১০০০ টাকার বেশি হতে দেয় না।

৩.১৭ অনুশীলনীর সমাধান

সমাধান ১ — যথাক্রমে `2`, `8` এবং `10`। ১৭-কে ৫ দিয়ে ভাগ করলে ২ অবশিষ্ট থাকে। দ্বিতীয়টিতে গুণ আগে হয়; তৃতীয়টিতে বন্ধনী আগে।

সমাধান ২

is-even.js · JavaScript — চলবে: Browser Console ও Node.js

```
function isEven(number) {
  return number % 2 === 0;
}

console.log(isEven(10));
console.log(isEven(7));
```

Output

```
true
false
```

সমাধান ৩

points.js · JavaScript — চলবে: Browser Console ও Node.js

```
let points = 50;

points += 10;
console.log(points);

points *= 2;
console.log(points);

points--;
console.log(points);
```

Output

```
60
120
119
```

সমাধান ৪ — Truthy: `'0'`, `' '` এবং `'null'`, কারণ তিনটিই ফাঁকা নয় এমন String। Falsy: `0`, `''` এবং `null`।

সমাধান ৫ — `age >= 18 || age < 18` যেকোনো সংখ্যার জন্য সত্য, কারণ দুটির একটি সব সময় মিলবেই। `||`-এর পরের অংশটি বাদ দিতে হবে।

can-vote.js · JavaScript — চলবে: Browser Console ও Node.js

```
function canVote(age) {
  return age >= 18;
}

console.log(canVote(15));
console.log(canVote(18));
```

Output

```
false
true
```

সমাধান ৬

get-grade.js · JavaScript — চলবে: Browser Console ও Node.js

```
function getGrade(mark) {
  if (mark >= 80) {
    return 'A+';
  } else if (mark >= 60) {
    return 'A';
  } else if (mark >= 33) {
    return 'B';
  } else {
    return 'F';
  }
}

console.log(getGrade(80));
console.log(getGrade(79));
console.log(getGrade(33));
console.log(getGrade(32));
```

Output

```
A+
A
B
F
```

সমাধান ৭

season.js · JavaScript — চলবে: Browser Console ও Node.js

```
function seasonOf(month) {
  switch (month) {
    case 12:
    case 1:
    case 2:
      return 'শীত';
    case 3:
    case 4:
    case 5:
      return 'গ্রীষ্ম';
    case 6:
    case 7:
    case 8:
    case 9:
      return 'বর্ষা';
    case 10:
    case 11:
      return 'হেমন্ত';
    default:
      return 'অজানা';
  }
}

console.log(seasonOf(1));
console.log(seasonOf(7));
console.log(seasonOf(13));
```

Output

```
শীত
বর্ষা
অজানা
```

সমাধান ৮

ticket.js · JavaScript — চলবে: Browser Console ও Node.js

```
function ticketPrice(age, isMember) {
  let price = 100;

  if (age < 5) {
    price = 0;
  } else if (age <= 12) {
    price = 50;
  }

  return isMember ? price / 2 : price;
}

console.log(ticketPrice(4, false));
console.log(ticketPrice(12, false));
console.log(ticketPrice(30, true));
```

Output

```
0
50
50
```

সমাধান ৯

valid-age.js · JavaScript — চলবে: Browser Console ও Node.js

```
function isValidAge(text) {
  const clean = text.trim();

  if (clean === '') {
    return false;
  }

  const age = Number(clean);

  return Number.isInteger(age) && age >= 0 && age <= 120;
}

console.log(isValidAge('25'));
console.log(isValidAge('0'));
console.log(isValidAge('121'));
console.log(isValidAge(''));
```

Output

```
true
true
false
false
```

সমাধান ১০

flat200.js · JavaScript — চলবে: Browser Console ও Node.js

```js
function flat200Discount(subtotal) {
  return subtotal >= 1000 ? 200 : 0;
}

console.log(flat200Discount(999));
console.log(flat200Discount(1000));
```

Output

```
0
200
```

সমাধান ১১

delivery-fee.js · JavaScript — চলবে: Browser Console ও Node.js

```js
function deliveryFee(amount) {
  if (amount >= 3000) {
    return 0;
  } else if (amount >= 1500) {
    return 60;
  } else {
    return 120;
  }
}

console.log(deliveryFee(3000));
console.log(deliveryFee(2999));
console.log(deliveryFee(1500));
console.log(deliveryFee(1499));
```

Output

```
0
60
60
120
```

সমাধান ১২

capped-discount.js · JavaScript — চলবে: Browser Console ও Node.js

```js
const MAX_DISCOUNT = 1000;

function cappedDiscount(subtotal, percent) {
  const discount = Math.round(subtotal * percent / 100);

  return discount > MAX_DISCOUNT ? MAX_DISCOUNT : discount;
}

console.log(cappedDiscount(3000, 10));
console.log(cappedDiscount(20000, 10));
```

Output

```
300
1000
```

৩.১৮ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. `17 % 5`-এর ফল কত?

ক) `3`

খ) `2`

গ) `3.4`

ঘ) `12`

প্রশ্ন ২. `let a = 5; console.log(a++);` কী দেখাবে?

ক) `6`

খ) `4`

গ) `5`

ঘ) একটি Error

প্রশ্ন ৩. `'10' < '9'`-এর ফল কী?

ক) `true`, কারণ String অক্ষর ধরে তুলনা হয়

খ) `false`, কারণ ১০ বড়

গ) `NaN`

ঘ) একটি Error

প্রশ্ন ৪. নিচের কোনটি Falsy?

ক) `'0'`

খ) `' '`

গ) `'false'`

ঘ) `0`

প্রশ্ন ৫. `true || false && false`-এর ফল কী?

ক) `false`

খ) `true`

গ) `undefined`

ঘ) একটি Error

প্রশ্ন ৬. `0 ?? 10`-এর ফল কী?

ক) `10`

খ) `null`

গ) `0`

ঘ) `false`

প্রশ্ন ৭. `else if`-এর একাধিক শর্ত সত্য হলে কোন Block চলে?

ক) সবগুলো

খ) শেষ যেটি সত্য

গ) কোনোটিই নয়

ঘ) প্রথম যেটি সত্য

প্রশ্ন ৮. `switch`-এ একটি `case`-এর শেষে `break` বা `return` না থাকলে কী হয়?

ক) পরের `case`-এর Code-ও চলে

খ) একটি Error হয়

গ) `default` চলে

ঘ) `switch` থেকে বেরিয়ে যায়

> **প্রশ্ন ৯.** `const fee = total >= 3000 ? 0 : 120;`-তে `total` যদি `3000` হয়, `fee` কত?
>
> ক) `120`
>
> খ) `3000`
>
> গ) `0`
>
> ঘ) `undefined`

> **প্রশ্ন ১০.** `if (coupon = 'SAVE10')` লিখলে কী ঘটে?
>
> ক) `coupon`-এর সঙ্গে `'SAVE10'` তুলনা হয়
>
> খ) `coupon`-এ `'SAVE10'` রাখা হয় এবং শর্ত সত্য হয়
>
> গ) সব সময় একটি Error হয়
>
> ঘ) শর্ত সব সময় মিথ্যা হয়

৩.১৯ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–গ · ৩–ক · ৪–ঘ · ৫–খ · ৬–গ · ৭–ঘ · ৮–ক · ৯–গ · ১০–খ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	১৭-কে ৫ দিয়ে ভাগ করলে ভাগফল ৩ এবং ভাগশেষ ২। `%` ভাগশেষ দেয়।
২	গ	`a++` আগে পুরোনো মান দেয়, তারপর বাড়ায়। এরপর `a`-এর মান হয় `6`।
৩	ক	`'1'` অভিধানে `'9'`-এর আগে আসে। সংখ্যা হিসেবে তুলনা করতে আগে `Number()` দরকার।
৪	ঘ	বাকি তিনটি ফাঁকা নয় এমন String, তাই Truthy।
৫	খ	`&&` আগে হিসাব হয়: `false && false` হলো `false`; তারপর `true || false` হলো `true`।
৬	গ	`??` শুধু `null` বা `undefined` হলে বিকল্প নেয়। `0` একটি বৈধ মান হিসেবে থেকে যায়।
৭	ঘ	শর্তগুলো ওপর থেকে নিচে যাচাই হয়; প্রথম সত্যটির Block চালিয়ে বাকিগুলো বাদ দেওয়া হয়।
৮	ক	একে Fallthrough বলে। ইচ্ছাকৃত না হলে এটি একটি Bug।
৯	গ	`>=` মানে বড় অথবা সমান, তাই শর্ত সত্য এবং প্রথম মানটি (`0`) বেছে নেওয়া হয়।
১০	খ	একটি `=` হলো Assignment। String-টি Truthy, তাই শর্ত সব সময় সত্য হয়।

৩.২০ অধ্যায়ের সারসংক্ষেপ

- Expression থেকে একটি মান পাওয়া যায়; Operator মান নিয়ে কাজ করে।
- Arithmetic: `+`, `-`, `*`, `/`, `%`, `**`। ভাগশেষ দিয়ে জোড়-বিজোড় ও চক্রাকার হিসাব করা যায়।
- `+=`, `-=`, `++`, `--` Variable-এর মান সংক্ষেপে বদলায়; `++` ও `--` আলাদা লাইনে লিখুন।
- Comparison-এর ফল Boolean। সীমার ক্ষেত্রে `>` আর `>=`-এর পার্থক্য খেয়াল করুন; String-এর তুলনা অভিধানের নিয়মে হয়।
- `&&` দুই পাশই সত্য চায়, `||` যেকোনো একটি, `!` উল্টে দেয়। `&&` আগে হিসাব হয়।
- প্রধান Falsy মান ছয়টি: `false`, `0`, `''`, `null`, `undefined`, `NaN`। শূন্য বৈধ হলে `if (value)` ব্যবহার করবেন না।
- `else if`-এ প্রথম সত্য শর্তটিই চলে, তাই কঠোরতম শর্ত আগে।
- নির্দিষ্ট মান মেলাতে `switch`; প্রতিটি `case`-এ `break` বা `return`, এবং সব সময় `default`।
- সরল দুই-পথের সিদ্ধান্তে Ternary।
- সন্দেহ হলে বন্ধনী।
- যাচাইয়ের ক্রম: আছে কি না, Type, সীমা, তারপর অন্য মানের সঙ্গে নিয়ম।

পরের অধ্যায়ে Loop: একই কাজ বারবার করা। সেখানে একাধিক পণ্যের Cart হিসাব করা হবে।

৩.২১ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Expression	যে Code থেকে একটি মান পাওয়া যায়।
Operator / Operand	যে চিহ্ন কাজ করে / যে মানের ওপর কাজ হয়।
Remainder	ভাগ করার পর অবশিষ্ট অংশ (`%`)।
Comparison Operator	দুটি মান তুলনা করে Boolean দেয়।
Logical Operator	`&&`, `||`, `!`: শর্ত জোড়া লাগায় বা উল্টে দেয়।
Short-Circuit	ফল নিশ্চিত হয়ে গেলে বাকি অংশ হিসাব না করা।
Truthy / Falsy	শর্তে যে মান সত্য / মিথ্যা হিসেবে গণ্য হয়।
Fallthrough	`switch`-এ `break` না থাকায় পরের `case`-এ চলে যাওয়া।
Ternary Operator	`শর্ত ? মান১ : মান২`।
Operator Precedence	কোন Operator আগে হিসাব হবে, তার নিয়ম।
Input Validation	ব্যবহারের আগে বাইরে থেকে আসা মান যাচাই করা।

পরিভাষা	অর্থ
Guard Clause	Function-এর শুরুতে অগ্রহণযোগ্য অবস্থা ধরে `return` করা।

৩.২২ আরও পড়ুন

- **MDN — Expressions and operators:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Expressions_and_operators` — সব Operator-এর বিবরণ।
- **MDN — if...else:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/if...else` — শর্তযুক্ত Statement।
- **MDN — switch:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/switch` — `switch`, `break` এবং Fallthrough।
- **MDN — Operator precedence:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Operator_precedence` — পূর্ণ Precedence তালিকা।
- **MDN — Falsy:** `https://developer.mozilla.org/en-US/docs/Glossary/Falsy` — Falsy মানের সম্পূর্ণ তালিকা।